HANDS-ON PERTEMUAN 3

Keamanan Aplikasi Web

Pembaruan Teknologi dan Framework Keamanan untuk Aplikasi Web

Program Studi Teknologi Informasi - Institut Teknologi Tangerang Selatan
Tahun Akademik 2026/2027 Ganjil

Fokus praktikum

Mahasiswa menelusuri dokumentasi dan security advisory, membandingkan baseline dan secure implementation, melakukan checklist SAST manual, membaca dependency advisory, dan menyusun rekomendasi pembaruan keamanan berbasis evidence.

1. Tujuan Pembelajaran

• Menjelaskan perkembangan teknologi keamanan aplikasi web dan peran framework keamanan modern.

• Membandingkan fitur keamanan framework atau middleware berdasarkan dokumentasi dan evidence teknis.

• Membedakan fungsi SAST, DAST, dan Software Composition Analysis (SCA).

• Melakukan observasi keamanan dasar pada aplikasi lokal: secure headers, output escaping, dan dependency advisory.

• Menyusun rekomendasi update framework/security tooling secara kritis dan dapat diverifikasi.

Batasan etika

Seluruh aktivitas hanya boleh dilakukan pada aplikasi lokal yang disediakan dalam ZIP starter lab. Jangan mengarahkan command, browser, proxy, scanner, atau payload ke domain kampus, layanan publik, atau sistem pihak ketiga tanpa izin tertulis.

2. Skenario Praktikum

Anda diminta membantu tim pengembang menilai apakah sebuah aplikasi web sederhana sudah menggunakan praktik framework security yang memadai. Aplikasi memiliki dua versi: baseline dan secure. Tugas mahasiswa adalah membandingkan keduanya, mengumpulkan evidence, lalu menulis rekomendasi teknis.

Komponen	File / aktivitas	Tujuan
Baseline app	app-baseline.js	Menunjukkan aplikasi berjalan tetapi belum memiliki kontrol keamanan dasar.
Secure app	app-secure.js	Menunjukkan perbaikan melalui output escaping dan secure headers.
Header checker	tools/header_check.js	Membantu mengecek Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, dan Referrer-Policy.
Dependency sample	package.json dan advisory-sample.json	Latihan membaca dependency advisory dan melakukan prioritas remediation.
Worksheet	security-checklist.md	Tempat mencatat evidence dan jawaban mahasiswa.

3. Persiapan Environment

• Browser modern: Chrome, Edge, Firefox, atau browser lain yang memiliki Developer Tools.

• Terminal: Command Prompt, PowerShell, Terminal macOS/Linux, atau terminal di VS Code.

• Node.js versi LTS atau versi yang tersedia di lab. Jika Node.js tidak tersedia, dosen dapat menjalankan demo dan mahasiswa mengisi worksheet berdasarkan output demo.

• Text editor: VS Code, Notepad++, atau editor lain.

• Opsional: npm audit jika internet tersedia. Jika tidak tersedia, gunakan advisory-sample.json sebagai sumber offline.

```
node -v
cd week3_framework_security_lab
node app-baseline.js
```

Evidence 0

Ambil screenshot terminal yang menunjukkan baseline app berjalan pada http://localhost:3000.

4. Step 1 - Observasi Baseline App

Jalankan aplikasi baseline, lalu amati response HTTP. Tujuan tahap ini adalah membedakan aplikasi yang sekadar berfungsi dengan aplikasi yang memiliki kontrol keamanan dasar.

```
node app-baseline.js

# Terminal baru:
curl -i http://localhost:3000/
node tools/header_check.js http://localhost:3000/
```

Yang dicek	Pertanyaan observasi	Catatan mahasiswa
Content-Security-Policy	Apakah response membatasi sumber script/style/img?	Status: 200 (MISSING). Header Content-Security-Policy sepenuhnya tidak ada pada respons HTTP baseline app. Hal ini menyebabkan browser tidak membatasi sumber eksekusi script/style/image, sehingga aplikasi sangat rentan terhadap serangan Cross-Site Scripting (XSS) dan pemuatan resource berbahaya dari pihak ketiga.
X-Frame-Options	Apakah halaman dapat dicegah dari embedding/clickjacking?	Status: 200 (MISSING). Header X-Frame-Options tidak ditemukan pada respons HTTP baseline app. Tanpa header ini, halaman aplikasi dapat dimasukkan ke dalam <iframe> oleh situs eksternal, membuat pengguna rentan terhadap serangan clickjacking/UI redressing.
X-Content-Type-Options	Apakah browser dilarang melakukan content sniffing?	Status: 200 (MISSING). Header X-Content-Type-Options tidak dikonfigurasi (missing nosniff). Browser dapat mencoba menebak tipe konten (MIME type sniffing) dari file yang diunduh, yang berpotensi mengeksekusi file non-executable sebagai script berbahaya.
Referrer-Policy	Apakah kebocoran URL/parameter melalui Referer dikurangi?	Status: 200 (MISSING). Header Referrer-Policy tidak ditetapkan. Hal ini berisiko menyebabkan kebocoran URL sensitif beserta query parameter ke domain eksternal melalui header HTTP Referer saat pengguna mengklik tautan keluar.

Evidence 1

Simpan output curl -i dan output header_check.js untuk baseline app.

5. Step 2 - Observasi Rendering Data User

Aplikasi baseline sengaja tidak melakukan output escaping. Gunakan payload aman untuk melihat apakah input dianggap sebagai data atau sebagai kode.

```
http://localhost:3000/?q=<script>alert(1)</script>

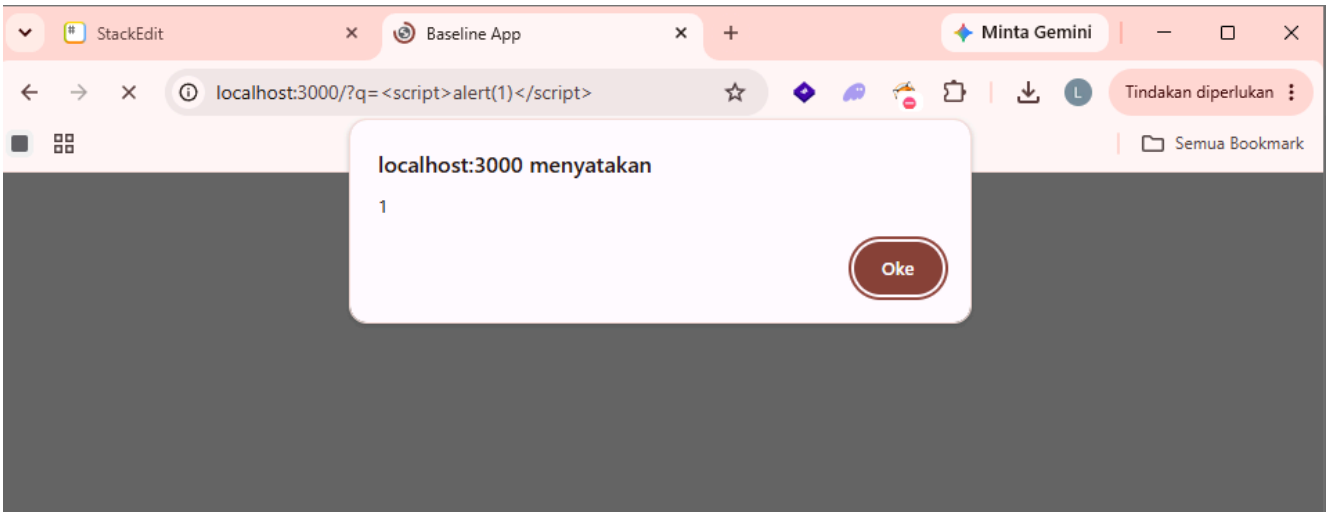
# Alternatif jika browser memblokir sebagian payload:
http://localhost:3000/?q=<b>halo</b>
```

- Jika payload HTML/JavaScript dieksekusi atau dirender sebagai markup, berarti output user belum diperlakukan aman sebagai data.
- Catat apakah response HTML menampilkan karakter < dan > secara literal atau sudah diubah menjadi < dan >.
- Kaitkan observasi ini dengan konsep framework auto-escaping atau templating yang aman.

Evidence 2

Ambil screenshot browser atau salin response HTML yang membuktikan perbedaan perilaku rendering.

Hasil:



6. Step 3 - Jalankan Versi Secure

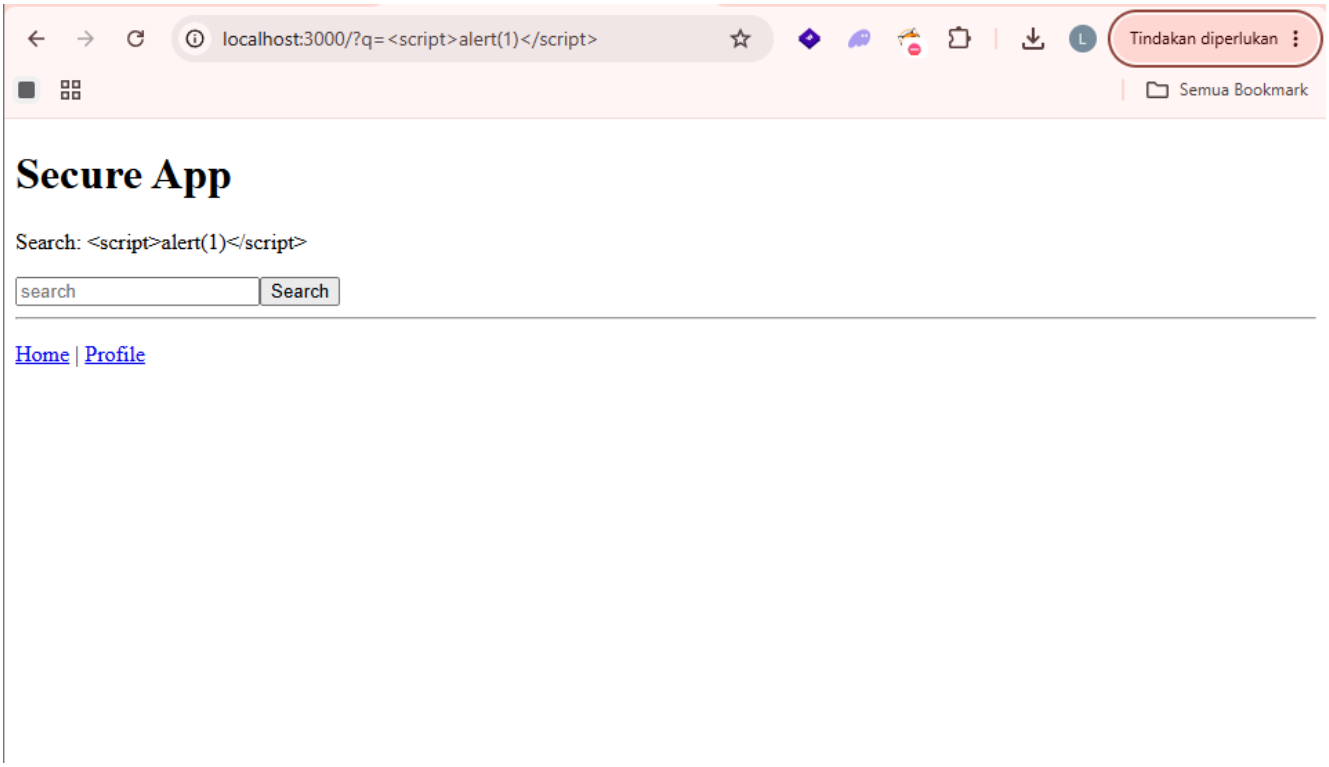
```
# Hentikan baseline dengan Ctrl+C
node app-secure.js

# Terminal baru:
curl -i http://localhost:3000/
node tools/header_check.js http://localhost:3000/

# Uji kembali payload:
http://localhost:3000/?q=<script>alert(1)</script>
```

Bandingkan hasil baseline dan secure. Versi secure menambahkan secure headers dan escape terhadap data user sebelum dirender ke HTML.

Hasil:



Aspek	Baseline	Secure
Secure headers	Header keamanan (CSP, X-Frame-Options, dll.) tidak ditemukan (Missing).	Secure headers aktif dikonfigurasi melalui middleware.
Output escaping	Tidak ada escaping; input user direnderlangsung sebagai HTML/JS.	Output di-escape secara otomatis sebelum direnderke HTML.
CSP	Tidak dikonfigurasi (Missing).	Dibatasi hanya untuk resource yang tepercaya.
Frame protection	Tidak dikonfigurasi; rentan clickjacking.	X-Frame-Options dikonfigurasi DENY/SAMEORIGIN.
Referrer policy	Tidak dikonfigurasi.	Ditetapkan ke no-referrer atau strict-origin-when-cross-origin.

7. Step 4 - Checklist SAST Manual

SAST tidak selalu harus dimulai dari tool otomatis. Dalam praktikum ini, mahasiswa melakukan review manual untuk mengidentifikasi source, sink, kontrol yang ada, dan kontrol yang hilang.

Pattern	Contoh pencarian	Risiko
Raw HTML	res.end(... userInput ...)	Reflected XSS / HTML injection
Dangerous sink	innerHTML, eval, Function	Code execution atau DOM XSS
SQL concat	"SELECT ..." + req.query	SQL Injection
Weak session	session id mudah ditebak	Session hijacking
Missing authorization	/admin tanpa cek role	Broken access control

```
File: app-baseline.js
Source: url.searchParams.get("q")
Sink: res.end(page(... q ...))
Kontrol hilang: output escaping dan secure headers
Rekomendasi: escapeHtml(), template auto-escaping, CSP, dan framework middleware
```

Evidence 3

Isi tabel SAST manual pada security-checklist.md. Minimal temukan satu source, satu sink, dan satu kontrol yang hilang pada baseline app.

8. Step 5 - Dependency Advisory Triage

Software Composition Analysis (SCA) memeriksa risiko dependency. Jika internet tersedia, gunakan npm audit. Jika tidak, gunakan file advisory-sample.json.

```
# Jika internet tersedia:
npm install
npm audit --audit-level=high

# Jika tidak tersedia:
cat advisory-sample.json
```

Package	Version	Severity	Impact	Fix	Priority
lodash	4.17.11	High	Prototype Pollution / ReDoS	Update ke >= 4.17.21	Tinggi

Triage yang baik

Jangan berhenti pada severity. Jelaskan apakah dependency reachable dari fitur aplikasi, apakah patch tersedia, apakah ada breaking change, dan bagaimana validasi setelah update.

9. Step 6 - Rekomendasi Pembaruan Berbasis Bukti

Setelah evidence terkumpul, susun rekomendasi teknis. Rekomendasi yang baik harus menjelaskan masalah, bukti, tindakan perbaikan, prioritas, dan cara validasi.

Bagian	Pertanyaan panduan	Contoh isi
Masalah	Kontrol apa yang hilang atau lemah?	CSP dan X-Frame-Options tidak ada pada baseline.
Bukti	Apa output command atau screenshot pendukung?	curl -i tidak menunjukkan header keamanan.
Dampak	Risiko apa yang meningkat?	XSS/clickjacking lebih mudah berdampak pada user.
Rekomendasi	Apa tindakan teknisnya?	Tambahkan middleware secure headers, output escaping, dan dependency update.
Validasi	Bagaimana membuktikan perbaikan?	Rerun header_check.js, test payload, npm audit, dan regression test.

10. Format Laporan Singkat

• Judul: Hands-On Pertemuan 3 - Framework Security Update Lab.

• Identitas: nama, NIM, kelas, tanggal praktikum.

• Ringkasan temuan maksimal 1 paragraf.

• Evidence: output curl, output header_check, screenshot atau potongan HTML, checklist SAST, dan dependency triage.

• Rekomendasi: minimal 3 tindakan teknis yang dapat diverifikasi.

• Refleksi: apa yang framework bantu otomatis, dan apa yang masih membutuhkan judgment developer.

Format pengumpulan

PDF atau Markdown maksimal 3 halaman. Gunakan bahasa teknis yang jelas dan sertakan evidence yang dapat direproduksi.

11. Rubrik Penilaian Hands-On

Komponen	Bobot	Kriteria
Evidence baseline vs secure	25%	Command/screenshot jelas, dapat direproduksi, dan relevan.
Analisis SAST manual	20%	Mampu mengidentifikasi source, sink, kontrol, dan rekomendasi.
Dependency triage	20%	Menjelaskan severity, impact, fix, dan prioritas dengan alasan.
Rekomendasi teknis	25%	Actionable, realistis, dan memiliki cara validasi.
Etika dan kerapian laporan	10%	Hanya localhost, tidak menyerang pihak ketiga, dan laporan rapi.

12. Troubleshooting

Masalah	Kemungkinan penyebab	Solusi
Port 3000 sudah dipakai	Ada proses Node lain berjalan	Matikan proses lama atau ubah PORT pada file app.
curl tidak dikenali	Windows lama atau PATH belum tersedia	Gunakan PowerShell Invoke-WebRequest atau browser DevTools Network.
npm audit gagal	Tidak ada internet atau npm registry tidak dapat diakses	Gunakan advisory-sample.json untuk mode offline.
Payload tidak memunculkan alert	Browser memblokir atau encoding berubah	Gunakan payload **halo** dan amati apakah markup dirender.
Header tidak berubah	Masih menjalankan baseline app	Ctrl+C lalu jalankan node app-secure.js.

13. Kuis dan Exit Ticket

Jawab singkat setelah praktikum:

• Apa perbedaan SAST, DAST, dan SCA?

• Apa dua kontrol keamanan yang biasanya disediakan framework modern?

• Mengapa security advisory perlu dibaca sebelum upgrade dependency?

• Apa bukti minimal bahwa secure headers sudah aktif?

• Mengapa framework aman tidak sama dengan aplikasi aman?

Exit ticket

Tuliskan satu hal yang sudah dipahami, satu hal yang masih membingungkan, dan satu kontrol keamanan yang ingin Anda dalami pada pertemuan berikutnya.

14. Referensi Singkat

• OWASP Top 10: https://top10.owasp.org/2021/

• OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/

• Django Security Documentation: https://docs.djangoproject.com/en/dev/topics/security/

• Spring Security Reference: https://docs.spring.io/spring-security/reference/

• GitHub Dependabot Alerts: https://docs.github.com/en/code-security/dependabot

• Materi semester terdahulu: DNS/HTTP, cookies/session, CSRF, XSS defenses, SQL Injection, SSRF, dan secure coding.

Lampiran A - Template Jawaban Mahasiswa

Pertanyaan	Jawaban mahasiswa
Perbedaan baseline vs secure headers	Adanya header seperti Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, dan Referrer-Policy pada HTTP response.
Payload rendering dan hasil observasi	Baseline tidak meng-escape input user sehingga payload HTML/JS dapat dieksekusi/direnderer, sedangkan secure meng-escape output.
Source dan sink yang ditemukan	Source: url.searchParams.get("q"), Sink: res.end(page(... q ...))
Dependency/advisory yang perlu diprioritaskan	Membaca security advisory untuk mengetahui severity, reachability, breaking changes, dan versi patch sebelum upgrade dependency.
Rekomendasi teknis utama	Menggunakan kontrol keamanan framework seperti output escaping otomatis, perlindungan CSRF bawaan, dan middleware secure headers.
Cara validasi perbaikan	Menjalankan header_check.js, menguji payload rendering, serta melakukan npm audit.
Refleksi akhir	Framework aman tidak sama dengan aplikasi aman karena keamanan aplikasi bergantung pada cara developer menulis logika, validasi input, dan konfigurasi yang tepat.

Lampiran B - Checklist Cepat Dosen

• Pastikan mahasiswa tidak mengganti target localhost menjadi website publik.

• Pastikan mahasiswa memahami bahwa missing header adalah risiko konfigurasi, bukan selalu vulnerability eksploit langsung.

• Tekankan bahwa framework security harus dibaca dari dokumentasi resmi dan advisory, bukan hanya blog/tutorial.

• Dorong mahasiswa menulis rekomendasi yang bisa diverifikasi ulang dengan command atau test.

• Gunakan kuis untuk mengecek pemahaman konseptual sebelum masuk ke materi DevSecOps pada pertemuan berikutnya.